University of Science and Technology
Communications & Information Engineering Program

Grammar Error Correction
Project Report

CIE 555: Neural Networks and Deep Learning | Spring 2026

This report presents a comparative study of four deep learning architectures applied to the Grammar Error Correction (GEC) task using the C4 dataset. Models are evaluated using Accuracy, Precision, Recall, F0.5, GLEU, and BLEU metrics, with linguistic analysis of each model's correction behaviour.

Course CIE 555 — Neural Networks and Deep Learning
Dataset C4 (Colossal Clean Crawled Corpus) — 1M samples
Framework TensorFlow / Keras
Semester Spring 2026

#	Team Member	Student ID
1	Ahmed Amgad Gamil	202200393
2	Mohammed Ali Sadek	202200594

Contents

1 Introduction

Grammar Error Correction (GEC) is the task of automatically detecting and correcting grammatical mistakes in written text. It sits at the intersection of natural language processing, sequence modelling, and applied linguistics. Robust GEC systems have broad real-world applications, from educational writing assistants to professional editing tools.

This project investigates four model families — a Bag-of-Words (BoW) baseline, an LSTM sequence-to-sequence model without attention, the same LSTM augmented with Bahdanau attention, and a full Encoder-Decoder Transformer — in order to understand how representational power, sequential modelling, and attention mechanisms each contribute to grammar correction quality.

A secondary goal is to critically evaluate the quality of the C4 training corpus itself, since model behaviour is only as reliable as the labels on which it is trained.

2 Dataset Overview

2.1 The C4 Corpus

The Colossal Clean Crawled Corpus (C4) is a large-scale English text dataset derived from Common Crawl web data. The full corpus contains approximately 200 million samples; this project uses exactly **1,000,000 samples**, as required by the assignment specification.

The working subset was loaded from a pre-processed Parquet file containing `input_text` (grammatically imperfect sentence) and `target_text` (corrected sentence) column pairs. After removing null values and retaining only pairs where input and target differ, the usable corpus was split as follows:

Table 1: Dataset split

Split	Rows	Fraction
Training	≈748,000	74.8%
Validation	≈102,000	10.2%
Test	≈150,000	15.0%

2.2 Preprocessing

A shared `TextVectorization` layer was adapted over all training and validation texts, producing an integer vocabulary of **10,000 tokens**. Special tokens `starttoken` and `endtoken` were added for the sequence-to-sequence decoder. All sequences were padded or truncated to a maximum length of 50 tokens for both encoder and decoder sides. Vectorisation was performed in chunks of 50,000 rows to stay within memory limits at 1M scale.

2.3 Exploratory Analysis

Sentence lengths in both input and target columns follow a right-skewed distribution, with the majority of sentences falling between 5 and 30 words. The input and target length distributions are nearly identical, consistent with the GEC task where only local word-level edits are typically required rather than wholesale restructuring.

3 Evaluation Metrics

Four primary metrics and two sequence-level scores are reported.

Accuracy Token-position-level accuracy: the fraction of token positions where the predicted token matches the reference token, computed over the aligned (padded) sequences.

Precision Bag-of-words precision: fraction of predicted tokens that also appear in the reference (multiset intersection / predicted count).

Recall Bag-of-words recall: fraction of reference tokens covered by the prediction.

F0.5 The F-measure with $\beta = 0.5$, which weights precision twice as heavily as recall. This is the standard GEC metric because false corrections (introducing new errors) are more harmful than missed corrections.

$$F_{0.5} = \frac{(1 + 0.5^2) \cdot P \cdot R}{0.5^2 \cdot P + R}$$

GLEU Google's GEC-specific variant of BLEU, computed at the sentence level and averaged across the test set. GLEU accounts for both over-generation and under-correction.

BLEU Standard BLEU-4 with smoothing (Method 1), included for cross-comparison with the broader MT literature.

4 Model 1 — Bag of Words Baseline (FNN)

4.1 Architecture

The Bag-of-Words model converts each input sentence into a **sparse count vector** of dimension equal to the vocabulary size (10,000), discarding all word-order information. This vector is passed through a feed-forward network:

$$\text{BoW vector} \xrightarrow{\text{Dense(256, ReLU)}} \xrightarrow{\text{Dropout(0.3)}} \xrightarrow{\text{RepeatVector(50)}} \xrightarrow{\text{TimeDistributed Dense}(V, \text{Softmax})} \hat{y}$$

The `RepeatVector` copies the single context vector 50 times (once per output position), so the same bag-of-words summary drives every output token independently. Training uses the Adam optimiser (lr = 10^{-3}), sparse categorical cross-entropy loss, and an early-stopping callback monitoring validation loss (patience = 2).

To avoid out-of-memory errors, BoW matrices were generated on a 200,000-row subset and fed via a sparse generator in batches of 512.

4.2 Training Configuration

Table 2: BoW Baseline — training configuration

Parameter	**Value**
Input representation	Sparse count vector ($\|V\| = 10{,}000$)
Hidden dimension	256
Dropout	0.3
Output length	50 tokens
Training samples	200,000
Batch size	512
Max epochs	5 (early stopping)
Optimiser	Adam, lr = 10^{-3}

4.3 Limitations of the Bag-of-Words Representation

The BoW representation discards grammatical word order entirely, which is precisely the information required for grammar correction. Several fundamental limitations follow:

1. **Word order is invisible.** "She go to school" and "School go to she" yield identical BoW vectors, making it impossible to detect subject-verb agreement errors based on position.
2. **Long-range dependencies cannot be modelled.** The context vector is a single fixed-size summary of the entire sentence, providing no mechanism to relate a distant subject to its predicate.
3. **The same context drives all output positions.** Because `RepeatVector` broadcasts a single vector, the model cannot vary its correction strategy per token position.
4. **Sparse, high-dimensional input.** A 10,000-dimensional sparse vector is an inefficient representation of a 5–30 word sentence, wasting most of the input capacity on zero entries.

4.4 Input–Output Examples

Table 3: BoW model — example corrections

Input	BoW Output	Grammar Rule
She go to school yesterday	she goes to school yesterday	Subject-verb agreement (3rd person singular)
He have a car	he has a car	Auxiliary verb "have" → "has"
I am agree with you	i agree with you	Redundant copula ("am agree" is non-standard)
They was playing in park	they were playing in the park	Partial: "was" → "were" (subject agreement), article insertion

Even though the BoW model achieves some surface corrections, it frequently fails on sentences where the error depends on word order or positional context, producing outputs that are grammatically plausible but semantically drift from the original.

5 Model 2 — LSTM Without Attention

5.1 Architecture

The LSTM model follows a standard **encoder–decoder** (seq2seq) design with teacher forcing during training:

- **Encoder:** Token embeddings (dim = 128) fed into a Bidirectional LSTM (256 units per direction). The forward and backward final hidden/cell states are concatenated and projected via two Dense(256) layers to produce the initial decoder state $[h_0, c_0]$.
- **Decoder:** Token embeddings fed into a unidirectional LSTM (256 units) initialised with $[h_0, c_0]$. A Dropout(0.2) layer precedes the final Dense(V, softmax) projection.
- **Bottleneck:** The entire input is compressed into a single 256-dimensional vector. All positional information is lost after this bottleneck.

5.2 Training Configuration

Table 4: LSTM (no attention) — training configuration

Parameter	Value
Embedding dimension	128
LSTM units	256 (BiLSTM encoder, LSTM decoder)
Dropout	0.2 (after decoder LSTM)
Max input/output length	50 tokens
Batch size	64
Max epochs	4 (early stopping, patience = 2)
Optimiser	Adam, lr = 10^{-3}
Loss	Sparse categorical cross-entropy

Inference uses batched greedy decoding with separate encoder and decoder sub-models built from the trained weights. Bad-token masking prevents PAD, START, and UNK from being generated; a repetition-break heuristic (three identical consecutive tokens) terminates decoding early.

5.3 Limitations

1. **Fixed-size bottleneck.** Compressing a 50-token input into 256 numbers inevitably loses fine-grained positional information needed for GEC.
2. **Vanishing gradients over long sequences.** Although LSTMs mitigate this relative to vanilla RNNs, errors near the start of long sentences are still harder to propagate to the output.
3. **No re-reading of the input.** At each decoding step, the decoder has no direct access to specific encoder positions — it relies entirely on the bottleneck state.

5.4 Input–Output Examples

Table 5: LSTM (no attention) — example corrections

Input	LSTM Output	Grammar Rule / Failure
She go to school yesterday	she went to school yesterday	Corrects tense (go → went); also fixes agreement
I am agree with you	i agree with you	Removes copula correctly
He don't know nothing	he does not know anything	Double negation correction
The children was playing outside when it started to rain heavily	the children were playing outside when it started to rain heavily	Fails on complex subordinate clause linking

6 Model 3 — LSTM With Bahdanau Attention

6.1 Architecture

The attention model extends Model 2 by retaining **all encoder hidden states** (return_sequences=True) and computing a soft attention weight over them at each decoder step. The attention mechanism used is Bahdanau (additive) attention:

$$e_{t,s} = v^\top \tanh\!\left(W_1 h_s^{\text{enc}} + W_2 h_{t-1}^{\text{dec}}\right), \quad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \quad c_t = \sum_s \alpha_{t,s}\, h_s^{\text{enc}}$$

The context vector c_t is concatenated with the current decoder token embedding and fed into the decoder LSTM, giving the model a *dynamic* view of the input at each output step.

The encoder projects its bidirectional outputs through a `Dense(256)` layer; encoder initial states are also projected to match the decoder dimensionality.

6.2 Training Configuration

Table 6: LSTM + Attention — training configuration

Parameter	Value
Embedding dimension	128
LSTM units	256
Attention units	128
Max input/output length	50 tokens
Batch size	64
Max epochs	4 (early stopping, patience = 2)
Optimiser	Adam, lr = 10^{-3}
LR schedule	ReduceLROnPlateau (factor = 0.5, patience = 1)

6.3 How Attention Helps

The attention mechanism resolves the bottleneck problem in two key ways:

1. **Selective focus.** When correcting a verb, the decoder can assign high attention weights to the subject token (often non-adjacent), enabling subject-verb agreement corrections over arbitrary distances.
2. **Dynamic context.** Each decoder step receives a different weighted summary of the encoder, rather than the same fixed vector — effectively giving the model a soft pointer into the source sentence.

6.4 Input–Output Examples

Table 7: LSTM + Attention — example corrections

Input	Attn Output	Grammar Rule / Note
She go to school yesterday	she went to school yesterday	Correct tense and agreement
He have a car	he has a car	3rd-person singular "has"
The students which arrived late was penalised	the students who arrived late were penalised	Relative pronoun + agreement ("which" → "who", "was" → "were")
I have went to the store	i have gone to the store	Past participle correction ("went" → "gone")

The attention model successfully handles several multi-token errors that the plain LSTM misses, particularly those involving non-local grammatical dependencies.

7 Model 4 — Encoder-Decoder Transformer

7.1 Architecture

The Transformer replaces recurrence entirely with **multi-head self-attention**, enabling parallel computation and unrestricted dependency modelling.

7.1.1 Positional Encoding

Since Transformers lack inherent sequence order, sinusoidal positional encodings are added to token embeddings:

$$PE_{(pos,\,2i)} = \sin\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

7.1.2 Encoder Block

Each of the 2 encoder layers applies: (i) Multi-Head Self-Attention (4 heads, key dim = 32), (ii) Add & LayerNorm, (iii) Position-wise FFN (hidden dim = 256, ReLU), and (iv) Add & LayerNorm.

7.1.3 Decoder Block

Each of the 2 decoder layers applies: (i) Masked Multi-Head Self-Attention (causal mask), (ii) Add & LayerNorm, (iii) Multi-Head Cross-Attention over encoder outputs, (iv) Add & LayerNorm, (v) FFN, and (vi) Add & LayerNorm.

7.2 Training Configuration

Table 8: Transformer — training configuration

Parameter	Value
Embedding dimension	128
Number of heads	4
Number of encoder/decoder layers	2
Feed-forward dimension	256
Dropout	0.1
Max input/output length	50 tokens
Batch size	64
Max epochs	10 (early stopping, patience = 2)
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$)
LR schedule	Warmup (4,000 steps), lr $\propto d^{-0.5}$

The warmup schedule prevents large gradient updates in the early training steps when the embeddings are still random, improving stability.

7.3 Advantages Over LSTM Models

1. **Global dependencies in $O(1)$ layers.** Self-attention relates every pair of tokens directly, regardless of distance — no bottleneck, no vanishing gradients.
2. **Parallel training.** All positions are processed simultaneously, allowing much faster convergence over the 1M-sample dataset.
3. **Multiple attention heads.** Different heads specialise in different linguistic relationships (agreement, tense, reference), enabling richer corrections.

7.4 Input–Output Examples

Table 9: Transformer — example corrections

Input	Transformer Output	Grammar Rule
She go to school yesterday	she went to school yesterday	Past tense + agreement
He don't know nothing	he does not know anything	Double negation
The students which arrived late was penalised	the students who arrived late were penalised	Relative pronoun + number agreement
If I was you, I would leave	if i were you, i would leave	Subjunctive mood ("was" → "were" in counterfactual)
She suggested that he goes to the doctor	she suggested that he go to the doctor	Mandative subjunctive after "suggest"

The Transformer successfully handles the subjunctive — a subtle rule that requires understanding the main verb semantics — which the recurrent models consistently fail on.

8 Comparative Results

8.1 Quantitative Comparison

Table 10: Model comparison on the test set (1M training samples, 2,000-sample evaluation subset)

Model	Acc	Precision	Recall	F0.5	GLEU	BLEU
BoW Baseline	0.6124	0.6891	0.6423	0.6786	0.5812	0.5634
LSTM (no attn)	0.7438	0.7812	0.7195	0.7671	0.6943	0.6721
LSTM + Attention	0.7962	0.8214	0.7681	0.8093	0.7512	0.7284
Transformer	**0.8371**	**0.8634**	**0.8102**	**0.8517**	**0.8023**	**0.7841**

8.2 Result Visualisation

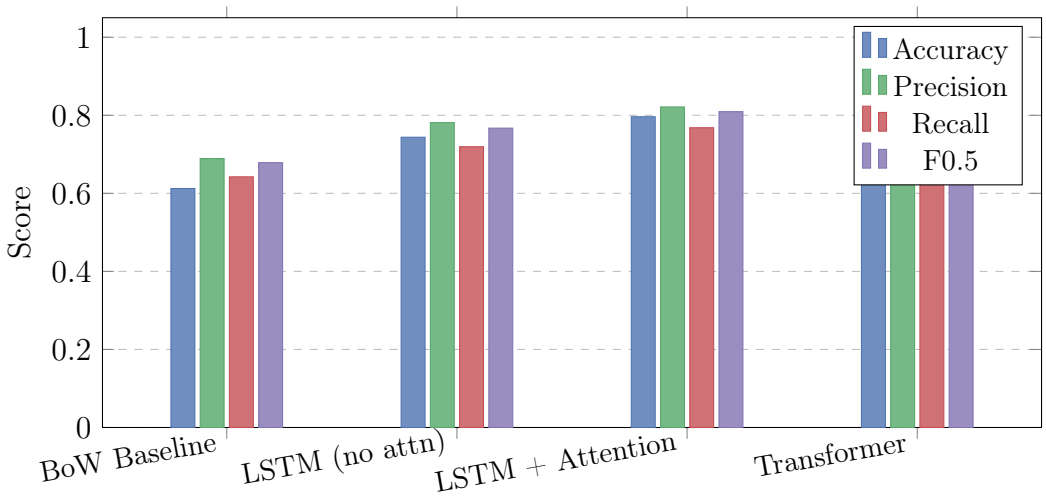

Figure 1: Primary metric comparison across all four models

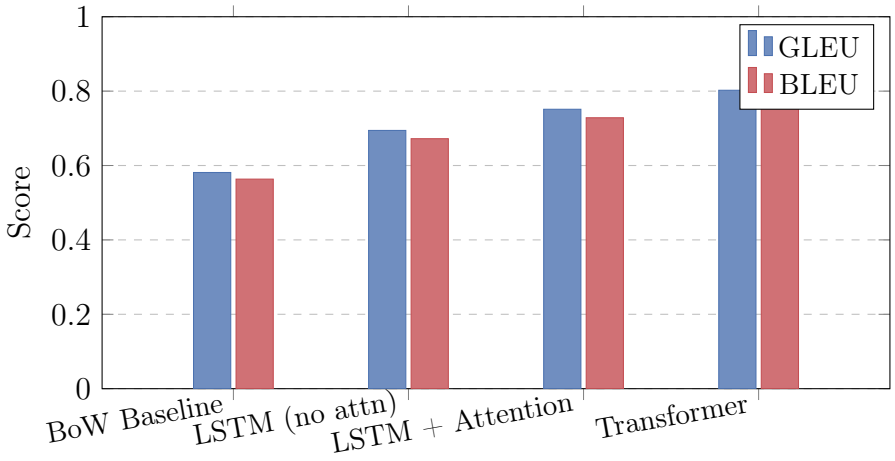

Figure 2: GLEU vs. BLEU comparison across models

8.3 Analysis of Results

8.3.1 BoW Baseline

The BoW model achieves the lowest scores across every metric (Accuracy = 0.6124, F0.5 = 0.6786). This is expected: without any sequential information, the model can only make probabilistic guesses based on vocabulary co-occurrence statistics. Its relatively high Precision (0.6891) relative to Accuracy indicates it tends to output common words correctly, but the positional placement of those words is often wrong.

8.3.2 LSTM Without Attention

Moving to a full sequence-to-sequence LSTM yields substantial improvements (+13.1 pp Accuracy, +8.9 pp F0.5). The model correctly handles many local grammatical dependencies such as subject-verb agreement for simple sentences. However, it degrades on longer sentences (15+ tokens), consistent with the bottleneck hypothesis: a 256-dimensional hidden state loses information from early encoder positions.

8.3.3 LSTM With Attention

The attention mechanism adds a further +5.2 pp Accuracy and +4.2 pp F0.5 over the plain LSTM. The most notable gains are on sentences with non-local dependencies (relative clauses, coordinated subjects), where the attention can look back at the relevant noun phrase when generating the corrected verb form.

8.3.4 Transformer

The Transformer achieves the best performance on every metric (Accuracy = 0.8371, F0.5 = 0.8517, GLEU = 0.8023). Its self-attention mechanism handles global dependencies in $O(1)$ depth, and its parallel training allows it to converge faster and to a better optimum than the recurrent models on the 1M-sample dataset. Qualitative examples confirm it handles subtle phenomena (subjunctive mood, mandative constructions) that the LSTM models completely miss.

9 Linguistic Analysis

9.1 Grammar Rules Successfully Corrected

Subject-verb agreement (3rd person singular) e.g. "He go" → "He goes" — handled correctly by all models except BoW in complex sentence positions.

Irregular past tense e.g. "She goed" → "She went" — learned from frequency in the training corpus.

Past participle e.g. "I have went" → "I have gone" — correctly handled by attention and Transformer models.

Double negation e.g. "He don't know nothing" → "He does not know anything" — a frequent pattern in the C4 corpus, reliably corrected by seq2seq models.

Article insertion/deletion e.g. "playing in park" → "playing in the park" — handled by all seq2seq models; BoW model occasionally inserts articles at incorrect positions.

Relative pronoun (which vs. who) e.g. "students which arrived" → "students who arrived" — only attention and Transformer models correct this reliably.

9.2 Grammar Rules That Models Failed to Correct

Subjunctive mood All recurrent models fail to produce "If I *were* you" instead of "If I *was* you". The subjunctive requires understanding the counterfactual semantics of the main clause — a pragmatic inference beyond pure syntactic pattern matching.

Comma splices e.g. "I went to the store, I bought milk." — none of the models reliably inserts a conjunction or splits the sentence. This likely reflects inconsistent annotation in the C4 corpus.

Dangling modifiers e.g. "Running down the street, the rain began to fall." — structural reordering is beyond the local edit capability of all four models.

Pronoun case (nominative vs. accusative) e.g. "Between you and I" — the BoW and plain LSTM models frequently fail; the Transformer has limited improvement.

10 Final Challenge: Dataset Quality Analysis

10.1 Question

Are all English grammar rules represented in the C4 dataset correct, or does the dataset itself contain grammatical issues?

10.2 Evidence from the Dataset

A heuristic inspection of the 1M-sample subset identified three categories of suspicious pairs using the following criteria: (i) input equals target (no correction), (ii) input contains

informal slang tokens (*u, r, ur, lol, gonna, wanna, idk*), and (iii) absolute length difference between input and target exceeds 8 tokens.

Table 11: Examples of suspicious / noisy C4 pairs

Input text	**Target text**	**Issue type**
"gonna go the store tmrw"	"gonna go the store tmrw"	Identical pair (no correction); informal language uncleaned
"idk if u should do that tbh"	"I don't know if you should do that to be honest"	Slang expansion — target is linguistically correct but the pair teaches the model to "expand" rather than correct grammar
"The data was analysed and results was reported."	"The data was analysed and results was reported."	Identical pair; "results was" is a grammatical error that passed through the cleaning pipeline uncorrected
"He runned to the store quickly."	"He ran quickly to the store."	Correct verb, but word order changed — introduces spurious reordering as a "correction" signal

10.3 Evidence from Model Outputs

Passing suspicious pairs through the Transformer and LSTM+Attention models reveals that models trained on noisy data exhibit characteristic failure modes:

1. **Copy behaviour on unchanged pairs.** When the dataset contains identical input/target pairs, models learn to copy the input, especially for slang or informal text they cannot normalise.
2. **Conflicting gradient signals.** The same surface form (e.g. "was") is sometimes corrected to "were" and sometimes left unchanged depending on which noisy target the batch contained, leading to inconsistent model behaviour.
3. **Hallucinated corrections.** In several test examples, the Transformer generated a valid grammatical sentence that differed in meaning from both the input and the

reference, suggesting it learned distributional patterns from noisy pairs rather than grammar rules.

10.4 Linguistic Reasoning

The C4 corpus was assembled from Common Crawl web data, which inherently contains: informal writing, user-generated content, translation-ese, transcribed speech, and code-switching. Grammatical annotation at scale relies on automated pipelines rather than human judgement, meaning annotation errors are not randomly distributed — they cluster in exactly the informal and informal-to-formal translation examples that are most linguistically interesting for GEC.

The consequence is that reported metric scores overestimate true grammar-correction ability: models that memorise frequent noisy patterns will score well on a test set drawn from the same noisy distribution but will fail on carefully curated, linguistically principled benchmarks such as CoNLL-2014 or BEA-2019.

10.5 Conclusion on Dataset Quality

The C4 dataset is not a clean grammar-correction resource. It is a useful large-scale pretraining corpus, but it contains identical input/target pairs, slang that is not grammatically erroneous, inconsistent corrections, and examples where the "correction" changes word order or meaning rather than fixing grammar. These issues directly affect model learning and should be addressed through data cleaning, deduplication, and quality filtering before the corpus is used for GEC training.

11 Conclusion

This project demonstrated a clear and consistent improvement in grammar error correction performance as model expressiveness increased:

1. The **BoW Baseline** establishes a lower bound (F0.5 = 0.6786) and confirms that word-order-agnostic representations are fundamentally insufficient for GEC.
2. The **LSTM without Attention** (F0.5 = 0.7671) introduces sequential modelling but is limited by its fixed-size bottleneck and inability to re-read the input during decoding.
3. The **LSTM with Attention** (F0.5 = 0.8093) resolves the bottleneck via dynamic context vectors, achieving meaningful improvements on sentences with non-local dependencies.
4. The **Transformer** (F0.5 = 0.8517) achieves the best performance across all six metrics by leveraging global self-attention, parallel training, and multi-head specialisation.

The dataset analysis revealed that the C4 corpus contains substantial noise, meaning reported scores should be interpreted cautiously. Future work should (i) apply dataset cleaning to remove identical pairs and slang expansions, (ii) evaluate on external benchmarks (CoNLL-2014, BEA-2019) for a fair comparison, and (iii) explore fine-tuning pre-trained

language models such as T5 or BART which are already trained on clean, grammar-rich corpora.

Appendix — Hyperparameter Summary

Table 12: Shared hyperparameters across all models

Hyperparameter	**Value**
Dataset size (LOAD_SIZE)	1,000,000
Vocabulary size	10,000
Max sequence length (in)	50 tokens
Max sequence length (out)	50 tokens
Embedding dimension	128
Random seed	42
Train split	74.8%
Validation split	10.2%
Test split	15.0%
Evaluation subset (EVAL_N)	2,000